1. Big Picture First (All 4 Methods)

All four approaches ultimately produce the same thing:

> A **computational graph in ONNX format**

But they differ in:

- how the model is represented internally
- how the graph is constructed
- how inputs are defined
- how weights are stored
- how deterministic the conversion is

2. High-Level Comparison Table

Framework	Export Tool	Model Type	Graph Source	Conversion Style	Complexity
PyTorch	torch.onnx.export	Dynamic eager model	Traced execution	Runtime tracing	Medium
TensorFlow	tf2onnx	Static graph model	Computation graph	Graph rewrite	Medium
sklearn	skl2onnx	Estimator object	Manual mapping	Operator conversion	Simple
sklearn Pipeline	skl2onnx	Multi-step pipeline	Chained estimators	Step-by-step graph build	Medium

3. PyTorch → ONNX

Method

torch.onnx.export(model, input, "model.onnx")

Library

PyTorch

Conversion style

Dynamic tracing

PyTorch runs the model once and records operations.

How it works internally

Input Tensor
↓
Forward pass execution
↓
Every operation is recorded (trace)
↓
ONNX graph is built from trace

Key characteristics

Pros:

- Very flexible
- Works with dynamic models
- Supports eager execution

Cons:

- Can miss control-flow logic
- Graph depends on sample input
- “What you trace is what you get”

Format behavior

- `.onnx` file
- sometimes `.onnx.data` (external weights)

4. TensorFlow / Keras → ONNX

Method

tf2onnx.convert.from_keras(model, ...)

Library

TensorFlow
tf2onnx

Conversion style

Static graph transformation

TensorFlow already has a computation graph.

How it works internally

Keras Model
↓
Computational Graph (TensorFlow graph)
↓
Node-by-node conversion to ONNX ops

Key characteristics

Pros:

- Stable conversion
- Fully graph-based
- Better optimization opportunities

Cons:

- Complex custom ops may fail
- Version/opset compatibility issues

Format behavior

- Usually **single `.onnx` file**
- weights embedded as ONNX initializers

5. scikit-learn → ONNX

Method

convert_sklearn(model, initial_types=...)

Library

skl2onnx
scikit-learn

Conversion style

Manual operator mapping

This is NOT tracing or graph export.

Instead:

> Each sklearn estimator is mapped to a predefined ONNX equivalent.

How it works internally

Example:

RandomForestClassifier
↓
TreeEnsembleClassifier (ONNX operator)

or

StandardScaler
↓
Sub + Div ONNX ops

Key characteristics

Pros:

- Very deterministic
- Fast conversion
- No tracing issues

Cons:

- Only supports known operators
- Custom models are hard to export

Format behavior

- Always `.onnx`
- No external `.data` usually needed

6. sklearn Pipeline → ONNX

Method

convert_sklearn(pipeline, initial_types)

Library

Same as sklearn ONNX:
skl2onnx

Conversion style

Sequential graph composition

Each pipeline step becomes a subgraph.

How it works internally

Example:

Input
↓
StandardScaler
↓
PCA
↓
LogisticRegression
↓
Output

ONNX builds:

- chained nodes
- intermediate tensor flow
- shared graph namespace

Key characteristics

Pros:

- Converts full ML workflow
- Includes preprocessing + model
- Production-ready pipelines

Cons:

- Debugging is harder
- Large graphs can become complex

Format behavior

- Single `.onnx`
- All steps embedded as graph nodes

7. Core Differences (Deep Insight)

A. How model is represented

Framework	Representation
PyTorch	execution trace
TensorFlow	static computation graph
sklearn	object → operator mapping
sklearn pipeline	chained object mapping

B. How ONNX graph is built

Framework	Graph Construction Method
PyTorch	runtime tracing
TensorFlow	graph conversion
sklearn	manual translation rules
pipeline	sequential composition

C. Weight handling

Framework	Weight storage
PyTorch	sometimes external `.onnx.data`
TensorFlow	embedded
sklearn	embedded
pipeline	embedded

D. Flexibility vs determinism

Framework	Flexibility	Determinism
PyTorch	⭐⭐⭐⭐⭐	⭐⭐
TensorFlow	⭐⭐⭐⭐	⭐⭐⭐⭐
sklearn	⭐⭐	⭐⭐⭐⭐⭐
pipeline	⭐⭐	⭐⭐⭐⭐⭐